EDUNEXUS V2

ANTIGRAVITY MASTER PROMPT

Multi-School Management SaaS Platform
Combined Product Specification + Implementation Prompt

Existing backend repository	school-backend
Existing frontend repository	school-application
Primary database	MySQL on Hostinger
Supporting integrations	Google Sheets API · WhatsApp Business Platform
UI direction	Professional glassmorphism · Animated sliding navigation

Purpose: Upgrade an existing single-school application into a secure, multi-tenant platform without destroying working code, existing repositories, user accounts or historical data.

Use this document as the master instruction in Google Antigravity.

How to Use This Prompt

Paste the full implementation prompt in this document into Antigravity after opening or checking out both existing repositories. If Antigravity cannot access them, provide the repository folders or connect the repositories first.

Critical instructions

- This is Version 2 of an existing application, not a greenfield replacement.
- Keep the backend and frontend in separate GitHub repositories.
- Inspect server.js, database configuration, routes, existing UI and Git status before editing.
- Do not delete, reset or overwrite production data. Back up the database before migrations.
- Build and test the backend, database and frontend together in incremental phases.
- Do not claim integrations, tests or deployments work until they have been verified.

Product Overview

Product name: EduNexus V2 — Multi-School Management Platform.

A central Super Admin manages multiple schools. Each school has its own School Admin, Sub-Admins, staff, teachers, students and parents. Every school sees only its authorized data. The platform manages onboarding, admissions, academics, timetable, attendance, homework, exams, marks, reports and parent notifications.

Main user portals

Portal	Core responsibilities
Super Admin	Global school onboarding, school status, platform analytics, global access control, audit logs and integration health.
School Admin	School profile, classes, sections, staff, students, guardians, timetable, exams, marks and school-level roles.
School Sub-Admin	Only the modules and class/department scopes granted by the School Admin.
Teacher / Staff	Assigned timetable, attendance, assigned subjects, homework, marks and authorized student records.
Student	Own profile where enabled, published timetable, homework, attendance and published academic results.
Parent / Guardian	Only linked child or children, timetable, attendance, homework, marks, reports and school announcements.

System Architecture

- **school-application:** React frontend, role-based portals, navigation, forms, charts and API client.
- **school-backend:** Existing Node.js/Express application, server.js or current entry point, APIs, services, authentication, RBAC, tenant isolation and integrations.
- **MySQL on Hostinger:** Primary source of truth for application records.
- **Google Sheets:** Controlled imports, exports and approved reporting; not a second authoritative database.
- **WhatsApp Business Platform:** Approved parent notifications and delivery status tracking.
- **PDF service:** Secure server-side generation of student reports, report cards and timetables.

Recommended frontend stack: React, TypeScript, Vite, Tailwind CSS, shadcn/ui, React Router, TanStack Query, React Hook Form, Zod and Recharts. Recommended backend stack: Node.js, Express, TypeScript, existing database driver or Prisma where compatible, validation, secure authentication and structured logging. Inspect the current stack first and preserve compatible working choices.

Master Antigravity Implementation Prompt

Copy the complete specification below into Antigravity. Execute it in phases and keep the two repositories separate.

1. Role and Objective

Act as a senior software architect, full-stack engineer, database designer, security engineer and UI/UX designer. Upgrade the existing single-school application into a production-oriented multi-tenant School Management SaaS platform. This must be a functional application, not only a static dashboard or mockup.

2. Existing Repository Context

There are two existing GitHub repositories:

- **school-backend:** Existing Node.js backend, server.js or current entry point, database connection, API routes, controllers, services and related modules.
- **school-application:** Existing frontend, UI components, routing, pages, API integration and school management features.

Inspect both repositories, their Git status, package files, source structure, authentication, schema, API routes and deployment configuration before editing. If the repositories are unavailable in the workspace, ask for access or checked-out folders. Do not invent missing files. Do not merge the repositories. Do not delete working functionality, Git history, environment configuration or existing database tables.

3. Repository Responsibilities

- **Backend repository:** APIs, database connection, migrations, models/data access, controllers, services, authentication, authorization, tenant isolation, audit logs, PDF generation, Google Sheets, WhatsApp and notification jobs.
- **Frontend repository:** React app, role-specific routes, design system, glassmorphic UI, animated menus, dashboards, forms, timetable pages, parent portal, charts, validation and API client.
- Each repository must retain its own README, environment example, setup instructions and tests. Never put backend secrets or database credentials in frontend code.

4. Mandatory Initial Inspection

1. Inspect frontend structure and existing routes.
2. Inspect server.js, middleware, routes, controllers and services.
3. Identify the current database engine and connection method.
4. Inspect tables, constraints, relationships and migration approach.
5. Review authentication, session/token handling and existing RBAC.
6. Map frontend pages to backend APIs.
7. Identify current features that must remain working.
8. Check Node.js/package versions and Hostinger plan constraints.
9. Review Git status and current uncommitted changes.
10. Create an implementation plan before editing.

Create docs/architecture.md, docs/database-design.md, docs/rbac-matrix.md, docs/api-specification.md, docs/migration-plan.md and docs/ui-design-system.md.

5. Multi-Tenant Security

- Create or extend schools with a stable UUID or equivalent internal ID, unique readable school code, school profile, status and timestamps.
- Associate each school-owned record with its tenant using reliable relationships and database constraints.
- Scope every protected query and mutation to the authenticated user's authorized school.
- Never trust school_id, user_id, role, class_id, student_id or other identifiers from the browser without validation.

- Prevent cross-school access through manipulated URLs, API bodies, query filters, exports, PDFs and uploaded-file links.
- Allow global Super Admin actions only through explicit authorization and audit logging.
- Write automated tenant-isolation tests proving that School A users cannot access School B data.

6. Super Admin Portal

- Login, global dashboard and database-backed metrics.
- School onboarding wizard, searchable school directory and school details.
- Activate, suspend and manage school records.
- Global user management and configurable platform roles.
- School-wise enrollment and staff analytics.
- Audit log viewer, Google Sheets sync monitoring and notification delivery summary.
- Platform settings and integration health.

7. Complete School Onboarding Wizard

Create a multi-step wizard with progress, Previous/Next navigation, Save Draft, inline validation, a review step and final confirmation.

Step 1 — School identity

- Official school name, short name, unique school code, logo, school type, management type, year established, registration/affiliation details where applicable.

Step 2 — Address and contact

- Address lines 1 and 2, locality, city, district, state, PIN code, country, official phone, email, website and optional alternate contact.

Step 3 — Academic information

- Board or affiliation, affiliation number where applicable, grades offered, medium of instruction, academic year, working days, school opening/closing times, period duration and break/lunch configuration.

Step 4 — School leadership

- Principal/head name, designation, official email and phone, initial School Admin invitation.

Step 5 — Configuration

- School branding, default classes/sections, timetable defaults, optional subscription plan/limits and final review.

Validate unique school codes. Mark optional fields clearly. Create the school and initial admin safely; avoid orphaned records. Use a secure invitation or one-time setup link instead of exposing a reusable password. Record onboarding in audit logs.

8. Student Admission and Registration

Build a multi-step student admission wizard with draft saving, clear validation, duplicate checks, review and an approval workflow where required.

Section	Fields / requirements
Personal identity	Admission number, legal name, preferred name if useful, date of birth, required demographic fields, photograph where appropriate.
Address	Current and permanent address, address lines, locality, city, district, state, PIN code, country, same-as-current checkbox.
Admission	Admission date, academic year, class, section, roll number, admission type, previous school/class and transfer certificate details where applicable.
Guardian 1	Full name, relationship, primary mobile, alternate number, email, address if different, contact preferences and authorization status.

Section	Fields / requirements
Guardian 2	Optional second guardian, relationship, mobile, email and communication preferences.
Emergency contact	Name, relationship and phone number.
Academic documents	Relevant records, optional supporting documents and private file storage.
Account linking	Link an existing verified parent account or send a secure parent invitation.

- Support multiple guardians per student and siblings under one verified parent account.
- Validate phone numbers and PIN codes without assuming all contacts are Indian.
- Use appropriate duplicate checks and school-scoped unique constraints.
- Generate admission numbers without exposing database IDs.
- Support draft, submitted, approved and rejected states as configured.
- Preserve academic-year enrollment history, promotion and transfer records.
- Allow authorized corrections and keep an audit trail.
- Collect only necessary data; restrict sensitive fields and documents.

9. Staff and User Management

Staff fields: staff ID, full name, official email, contact number, designation, department, employment type where needed, joining date, assigned subjects/classes and employment status. Keep staff registration separate from login activation. Support secure invitations, role assignment, password reset, session revocation, deactivation and audit history. Never store plaintext passwords.

10. RBAC and Permissions

Support Super Admin, School Admin, School Sub-Admin, Staff, Teacher, Student and Parent roles. Implement configurable permissions, school-specific role assignments, class/section restrictions and teacher subject assignments.

- Example permissions: schools.create, schools.update, students.read, students.create, students.update, staff.manage, attendance.record, attendance.correct, timetable.read, timetable.manage, timetable.publish, marks.enter, marks.approve, marks.publish, homework.manage, reports.download, notifications.manage and roles.manage.
- Adapt permission names to the existing application.
- Enforce permissions in backend middleware/services for every protected operation.
- Frontend menu hiding is only a usability feature, not a security boundary.
- Record role and permission changes in audit logs.

11. Timetable Management

Build a full timetable system, not a static timetable table. Support school, academic year, shift, working days, class, section, subject, teacher, classroom, period number, start/end time, break periods, effective date and publication status.

- Weekly calendar grid with day/week views and filters for class, section, teacher and subject.
- Create/edit periods, assign teachers and rooms, copy schedules between days or sections with validation.
- Optional drag-and-drop timetable builder, with conflict checks before saving.
- Draft, Published and Archived versions, preview, version history, printable timetable and PDF export.
- Prevent double-booking teachers and classrooms, overlapping periods for a class and scheduling outside configured school hours.
- Respect teacher availability, assigned subjects and school shifts.
- Use transactions or conflict-safe database operations when saving schedules.
- Notify affected users of published timetable changes where appropriate.

12. Student Timetable

- Automatically resolve the student's current active enrollment.
- Show only the published timetable for the student's school, academic year, class and section.
- Provide Today, Weekly, Subject List and Next Class views.
- Display subject, teacher, room, period start/end time and breaks.
- Support mobile-friendly schedule cards, printing and PDF export.
- Parents can view timetables only for their linked children.
- Students and parents cannot choose another class to access unauthorized schedules.

13. Teacher Timetable and School Configuration

- Teacher daily/weekly schedule, assigned classes, subject, period time, classroom, free periods and change announcements.
- School Admin configures working days, shifts, period templates, breaks, lunch, rooms and teacher availability.
- Allow authorized users to create, validate, preview, publish and archive timetable versions.
- Unapproved drafts must never appear in student or parent portals.

14. Academic Management

- Academic years, classes, sections, subjects and teacher assignments.
- Daily attendance, correction workflow and monthly reports.
- Exam creation, schedule, subjects, maximum marks and class-wise marks entry.
- Marks validation, draft/review/approval/publication and correction history.
- Homework and assignments, due dates, attachments and submission tracking.
- Restricted, factual behaviour records with appropriate access controls.
- Subject-wise trends, exam-to-exam progress, attendance trends and configurable follow-up alerts.
- Preserve historical academic records and audit changes to published marks.

15. Student Profile and PDF Reports

Build a student detail page with tabs for personal details, guardians, enrollment history, timetable, attendance, exams/marks, performance, homework, authorized behaviour records and documents.

Generate branded PDFs on the backend for suitable student profiles, report cards, attendance reports and timetables. Check authorization before generating or downloading. Use private storage or short-lived access-controlled links. Include school branding, student identifier, academic year and report date.

16. Parent Portal

- Secure login, verified parent-child linking and support for multiple children.
- Child profile, timetable, attendance, exam schedule, published marks, report cards, homework, assignments and announcements.
- Notification preferences and notification history.
- Parents can access only children linked to their verified account.

17. WhatsApp Business Integration

- Use the official WhatsApp Business Platform Cloud API.
- Support eligible homework, assignment deadline, timetable, exam, result, attendance and announcement notifications.
- Use approved templates where required and respect applicable consent/preferences.
- Implement a durable queue, retry limits, duplicate prevention, provider message IDs and delivery statuses.

- Keep API tokens in backend environment variables or a suitable secret manager.
- Do not claim delivery unless confirmed by provider status. The application must keep working when WhatsApp is unavailable.

18. Google Sheets Integration

- MySQL remains the authoritative database.
- Support bulk student/staff imports, approved exports and aggregate reporting.
- Preview and validate rows before import; show duplicates and row-level errors.
- Track sync history, retry safely and use stable IDs/idempotent operations.
- Enforce tenant scope on every import/export.
- Never place passwords, session tokens or unnecessary children's personal data in spreadsheets.

19. Professional Glassmorphic UI

Create a premium enterprise SaaS interface with refined glassmorphism, translucent panels, controlled backdrop blur, subtle gradients, soft borders/shadows, excellent typography, generous spacing and consistent iconography. Use transparency carefully to preserve readability and accessibility.

- Animated sliding sidebar with expand/collapse, smooth active-state transitions, role-specific menus and mobile drawer.
- Keyboard-accessible navigation, visible focus states and reduced-motion support.
- Responsive Super Admin, School Admin, Teacher, Student and Parent dashboards.
- Real database-backed KPI cards, enrollment/attendance charts, recent activity, upcoming exams, timetable summaries and pending approvals.
- Reusable tables with search, filters, sorting, pagination and export.
- Multi-step forms with progress indicators, inline validation, Save Draft and confirmation dialogs.
- Consistent loading, error, empty and success states.
- Avoid excessive blur, animation, gradients or low-contrast text. Prioritize usability, speed and accessibility.

20. Frontend Engineering

- Preserve the existing frontend framework and routing wherever practical.
- Implement a shared API client, environment-based API URL, centralized auth handling and role-aware navigation.
- Use protected routes as a UX layer, not a substitute for server authorization.
- Implement reusable forms, tables, dialogs, charts, toast notifications, pagination, filtering and accessible controls.
- Use query caching/invalidation and clear API error handling.
- Do not hardcode production metrics or fake successful responses.

21. Backend Engineering

- Preserve existing server.js or the current entry point and database configuration wherever possible.
- Implement tenant-scoped services, validation, authorization middleware, centralized error handling, transactions, pagination, logging and audit events.
- Handle files securely and isolate integration failures from core school operations.
- Use the existing database engine and compatible data-access layer after inspection.
- Never commit secrets or overwrite production environment files.

22. Security, Privacy and Quality

- Secure password hashing, session/token management, login rate limits and session revocation.
- Input validation, SQL injection prevention, secure HTTP headers and CSRF protection where applicable.

- Private document storage, least-privilege database credentials and environment-based secrets.
- Audit logs for sensitive actions, exports, downloads, permission changes and marks edits.
- Backups, restore tests, API pagination, database indexes and appropriate retention policies.
- Follow applicable Indian data-protection requirements and school policies, especially for children's data.
- Use fictional demo data only; never commit real student records to GitHub.

23. Testing and Acceptance Criteria

- Login/logout and account invitation tests.
- School onboarding and duplicate school-code tests.
- Student admission and guardian-linking tests.
- Staff invitation and RBAC tests.
- Cross-tenant isolation and parent-child access tests.
- Timetable conflict detection, publication and visibility tests.
- Attendance, marks approval and publication tests.
- PDF authorization tests.
- Google Sheets validation and WhatsApp retry tests.
- Regression tests for existing single-school features.

Run the relevant tests and report the actual results. Do not claim tests pass unless they were executed successfully.

24. Migration and Deployment

- Inspect the existing schema and create a backup before migration.
- Prepare incremental migrations and recoverable rollback steps where feasible.
- Map the original school into the new tenant model and preserve users, records and academic history.
- Never reset/drop production tables just to fit the new schema.
- Deploy backend and frontend from their respective repositories.
- Configure API URL, HTTPS, CORS allowlist, database credentials, auth secrets, Google credentials, WhatsApp credentials, private file storage, backups and health checks.
- Verify the selected Hostinger plan supports the required Node.js runtime, MySQL connectivity, file storage and background jobs.

25. Delivery Requirements

- Updated school-backend repository.
- Updated school-application repository.
- Database migrations and environment examples.
- API documentation, architecture, database and RBAC documentation.
- Automated tests and truthful execution results.
- Local setup instructions, demo-data script and Git workflow guidance.
- Hostinger deployment, migration verification and rollback instructions.
- Feature completion checklist and a list of any external credentials/approvals still required.

Work incrementally. Inspect first, preserve working features, implement each module across database/backend/frontend, run tests, and document unfinished dependencies. Do not stop after a plan or static UI. Deliver a functional, secure multi-school platform.

Recommended Build Order

1. **Phase 1:** Repository inspection, backup, architecture, migration plan and security tests.
2. **Phase 2:** Multi-tenant database model, authentication, RBAC and Super Admin school onboarding.
3. **Phase 3:** Complete student admission, parent/guardian linking, staff registration and school setup.
4. **Phase 4:** Timetable builder, teacher schedule, student timetable and conflict validation.
5. **Phase 5:** Attendance, homework, exams, marks, performance analytics and PDF reports.
6. **Phase 6:** Parent portal, WhatsApp and Google Sheets integrations.
7. **Phase 7:** UI polish, regression/security testing, staging, production deployment and monitoring.

Go-Live Checklist

- Both repositories have clean, understood changes and separate deployment instructions.
- Database backup and migration verification are complete.
- School A and School B tenant-isolation tests pass.
- Admin, teacher, student and parent access rules are verified.
- Timetable conflicts are blocked and only published schedules are visible to students/parents.
- Student PDFs and uploaded documents require authorization.
- Google and WhatsApp credentials are stored securely and integrations are tested where credentials are available.
- HTTPS, CORS, backups, logs, health checks and restore procedures are verified.
- Existing single-school features have passed regression tests.

Final instruction: Upgrade the existing application carefully. Keep school-backend and school-application separate. Prioritize tenant isolation and data preservation before adding more UI features. Build the complete functional product in tested phases.